

State Hoisting dan launched Effect

Nama : Muhammad Darren Achilles Setiawan

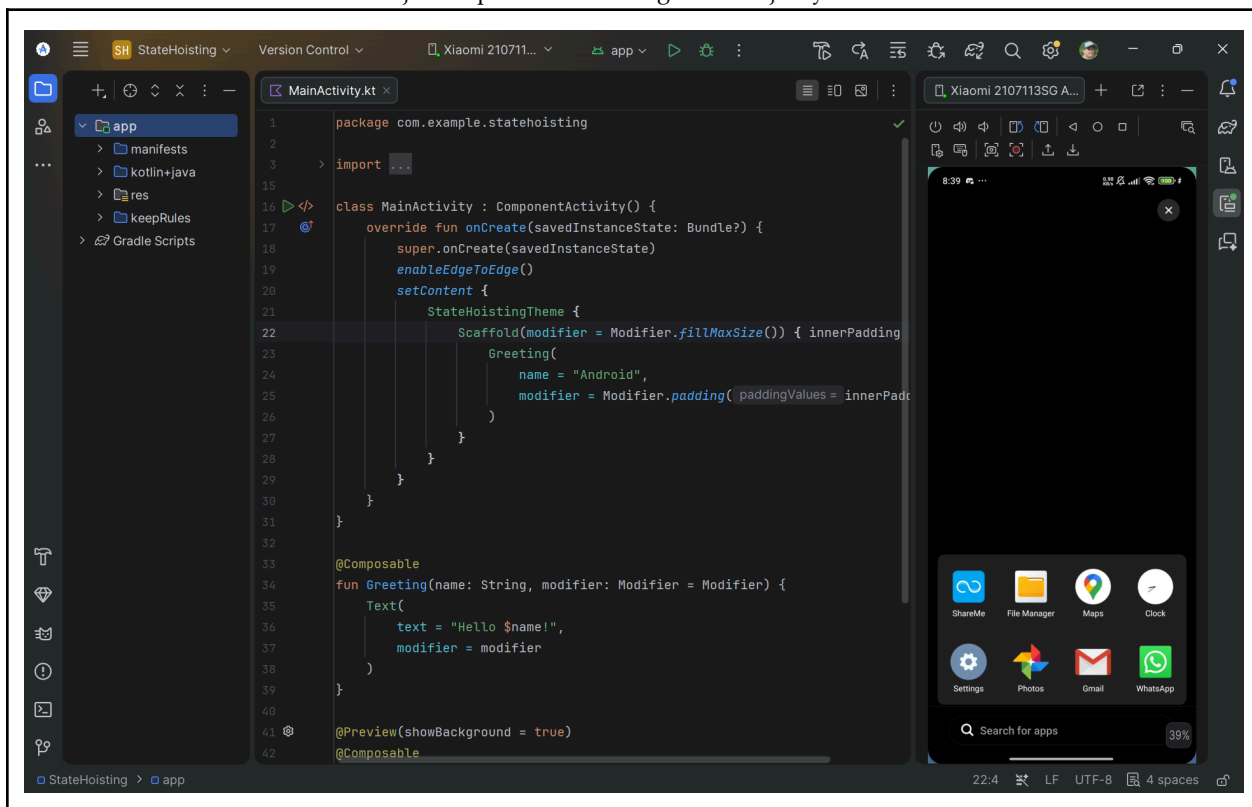
NIM : 245150401111049

Kelas : SI-F

1.1.1 State Hoisting

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template **Empty Activity** untuk **Phone and Tablet**. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.

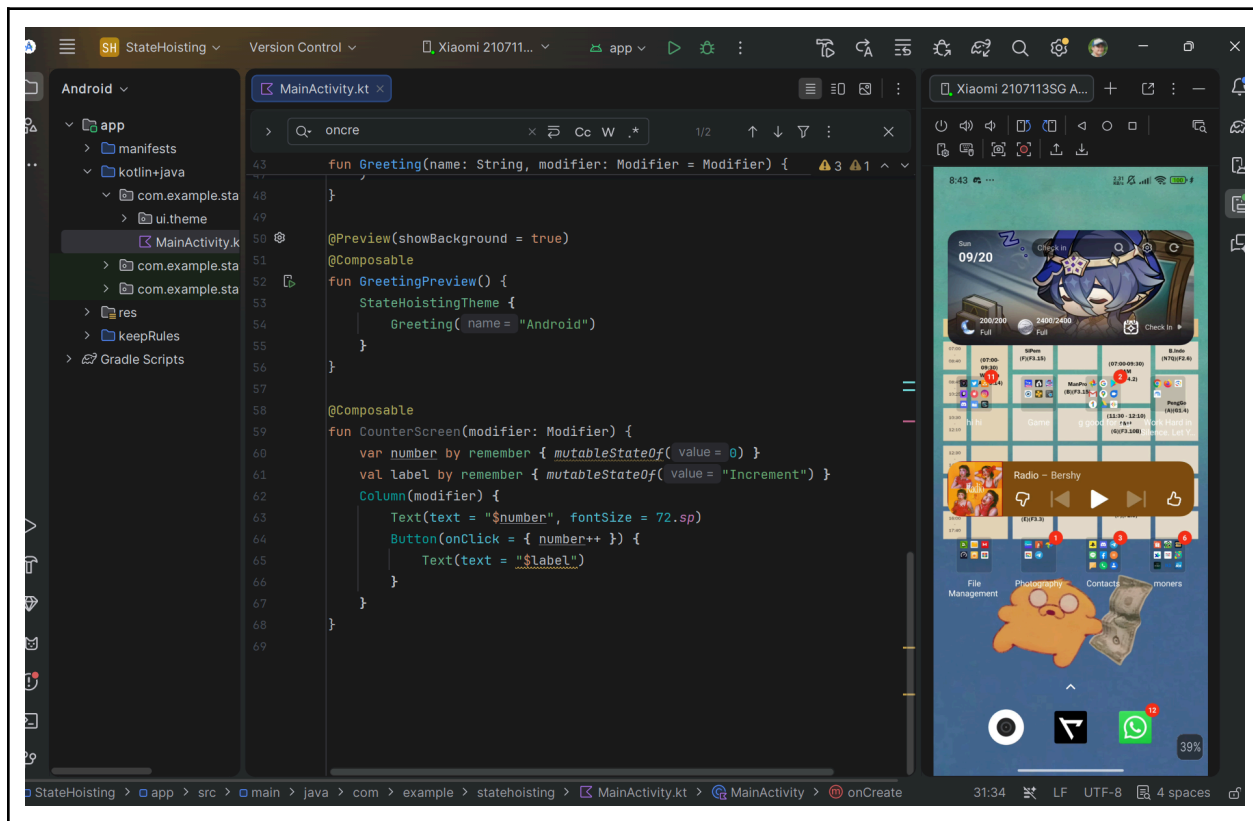


2. Buka file `MainActivity.kt` dan tambahkan fungsi composable berikut ini di **luar** class MainActivity:

```
fun CounterScreen(modifier: Modifier) {
    var number by remember { mutableStateOf(0) }
    val label by remember { mutableStateOf("Increment") }
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = { number++ }) {
            Text(text = "$label")
        }
    }
}
```

Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```
import androidx.compose.runtime.remember
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.material3.Button
import androidx.compose.foundation.layout.Column
    import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```



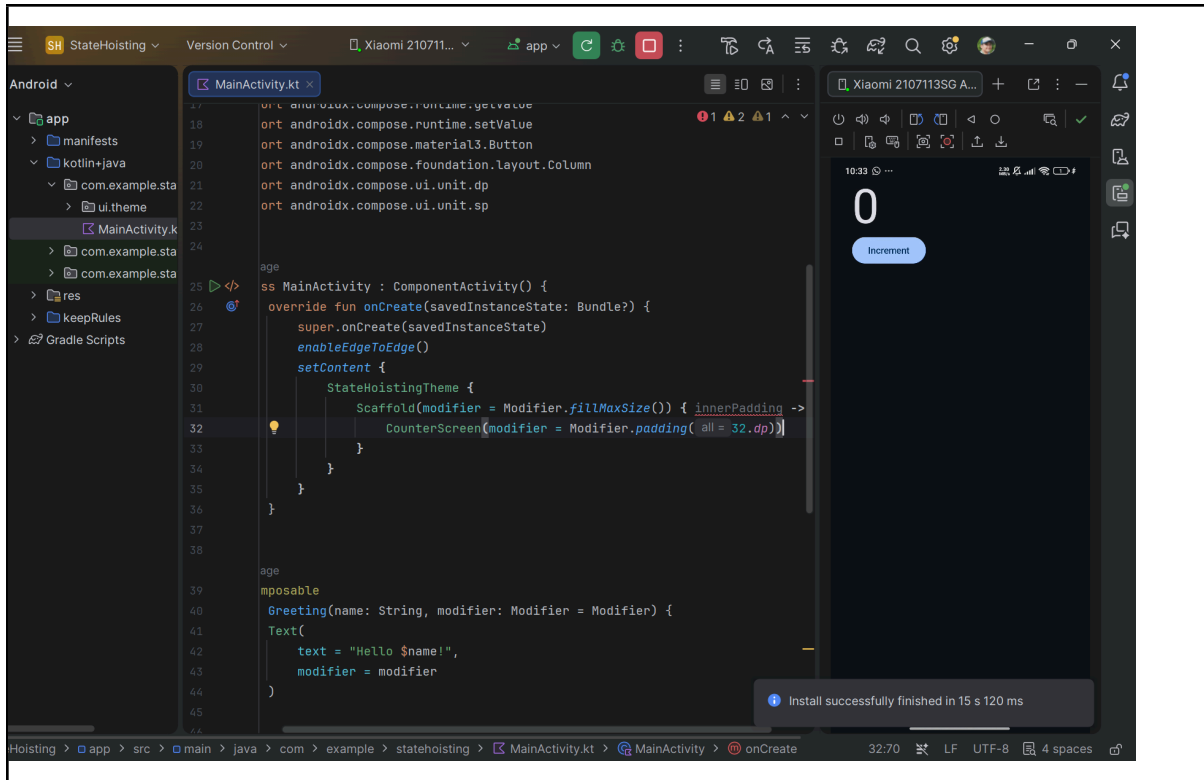
3. Buka file `MainActivity.kt`, temukan fungsi `onCreate()` yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah bagian kode program berikut di dalam fungsi `onCreate()`:

```
Greeting(
    name = "Android",
    modifier = Modifier.padding(innerPadding)
)
```

Kemudian ubah bagian tersebut menjadi:

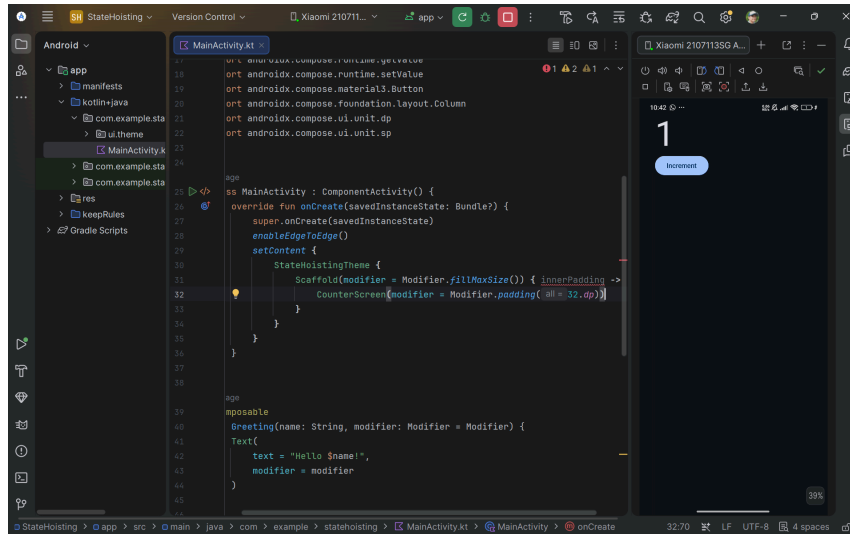
```
CounterScreen(modifier = Modifier.padding(32.dp))
```

Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.



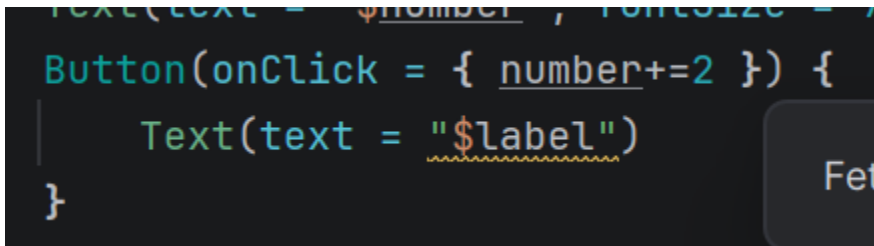
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.
 - a. Apa yang terjadi setiap kali tombol "Increment" diklik?
 - b. Apa yang menyebabkan hal tersebut terjadi?
 - c. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
 - d. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
- a. Angka yang ditampilkan pada layar bertambah 1.

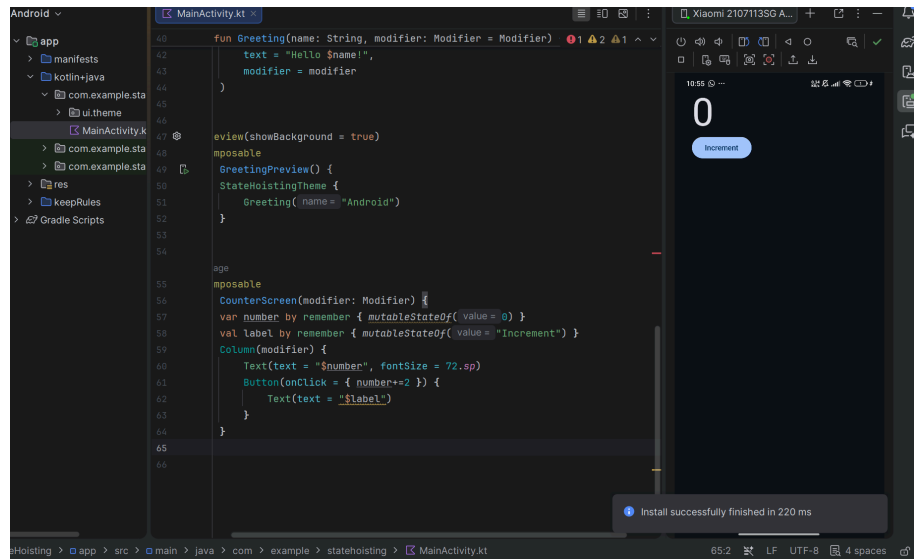


- b. Proses tersebut terjadi karena blok fungsi `onClick = { number++ }` dieksekusi. Hal ini memperbarui nilai variabel `state` dan memicu proses recomposisi pada `Composable Text`.
- c. Variabel `state` bernama `number`.
- d. Variabel `state` bernama `Label`

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable `CounterScreen` agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".
 - a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?
 - b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

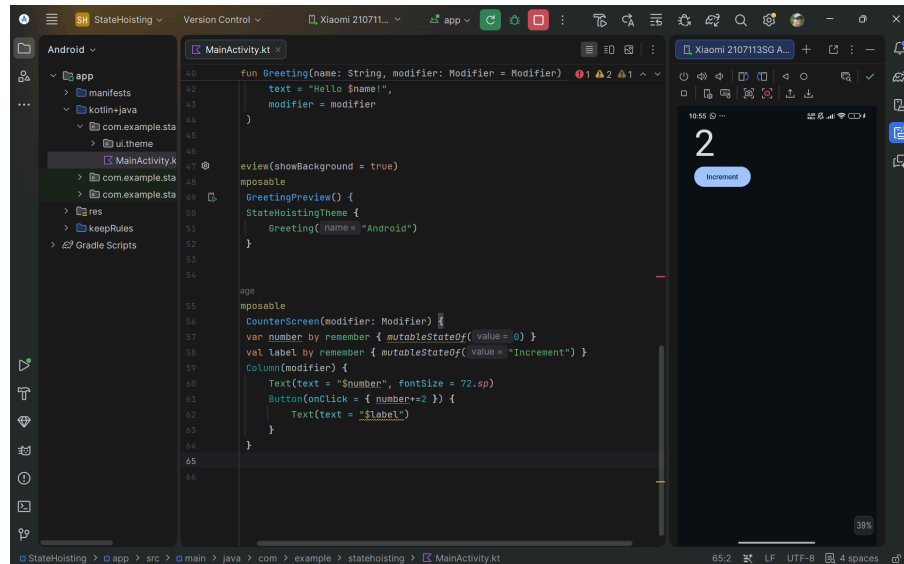


- a.
- b. Sebelum :



i.

Sesudah



ii.

- Langkah selanjutnya adalah melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable `CounterScreen`.

Ubah signature kode program pada bagian fungsi Composable `CounterScreen` dari yang sebelumnya:

```
fun CounterScreen(modifier: Modifier)
```

menjadi:

```
fun CounterScreen(modifier: Modifier,
    number: Int, label: String, onClick: () -> Unit)
```

Ubah *body* di dalam Composable `CounterScreen` menjadi:

```
// var number by remember { mutableStateOf(0) }
// val label by remember { mutableStateOf("Increment") }
Column(modifier) {
    Text(text = "$number", fontSize = 72.sp)
    Button(onClick = onClick) {
        Text(text = "$label")
    }
}
```

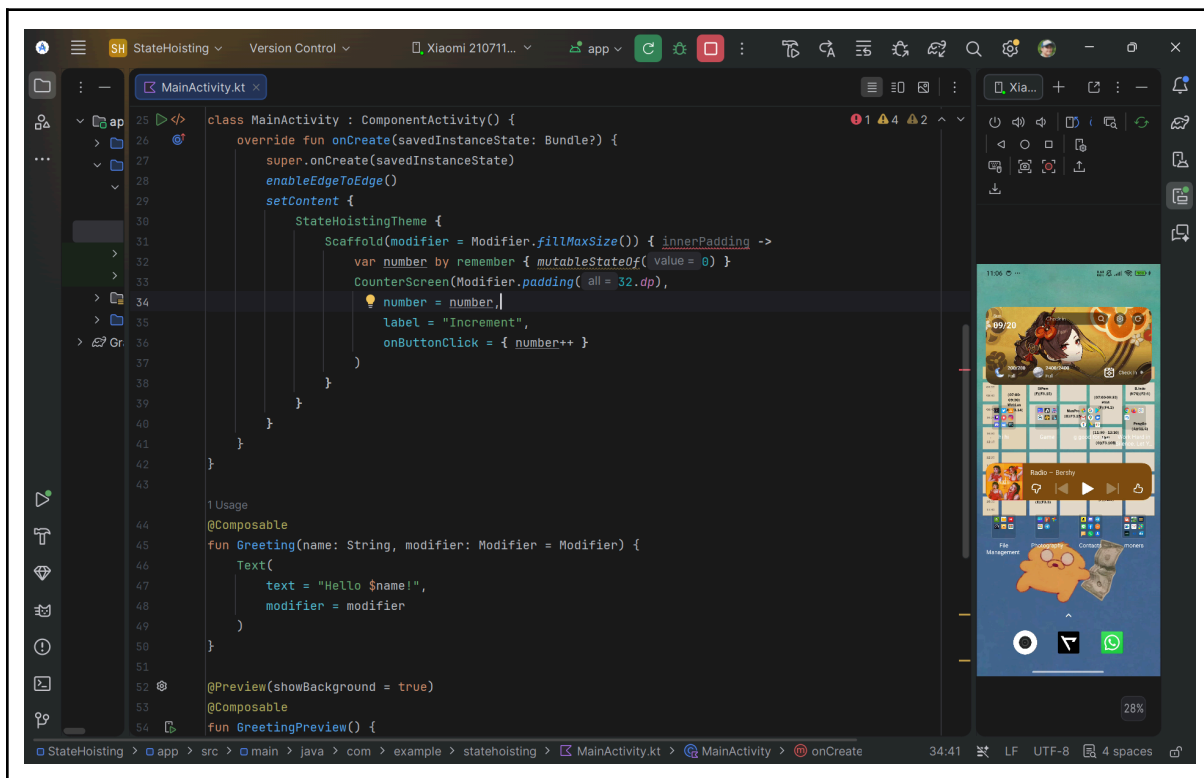
Ubah pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

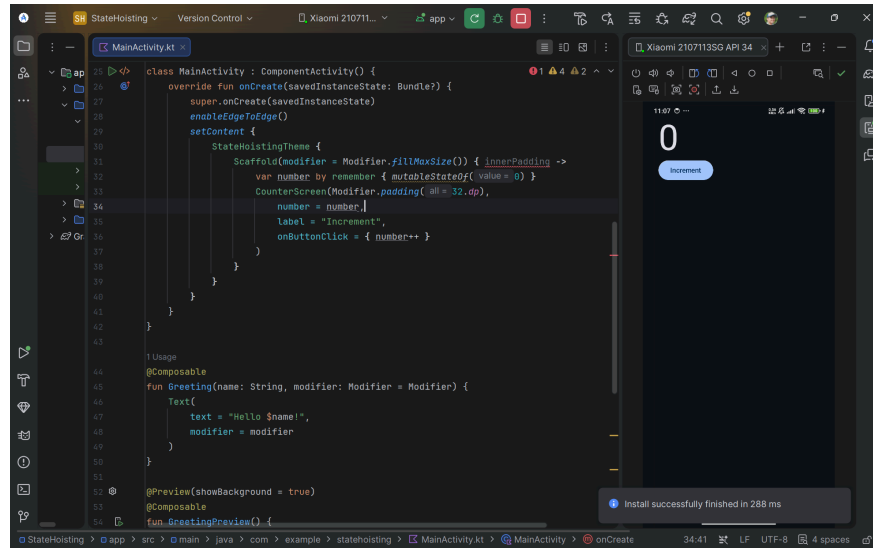
```
var number by remember { mutableStateOf(0) }
CounterScreen(Modifier.padding(32.dp),
    number = number,
    label = "Increment",
    onClick = { number++ }
)
```

Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().

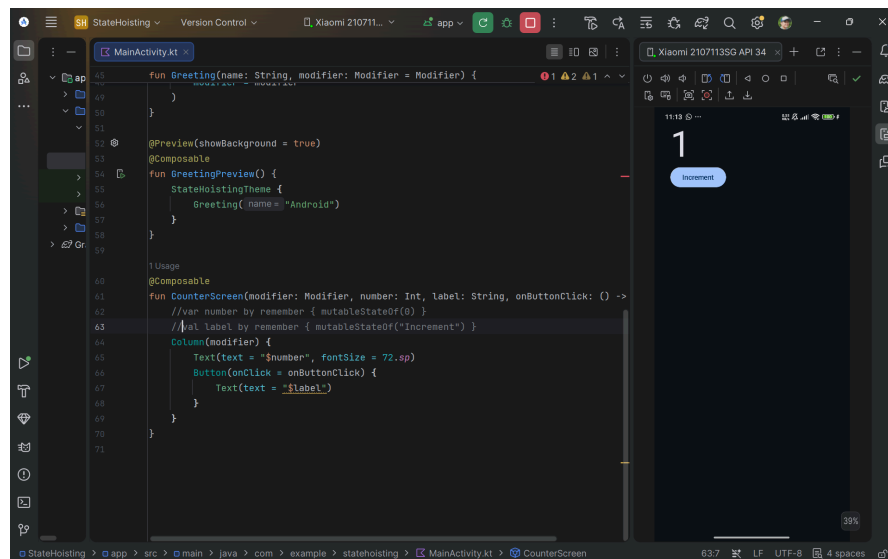


4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
 - a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
 - b. Di bagian kode program manakah saat ini proses increment dilakukan?
 - c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

a. Berjalan baik



i.



ii.

- b. Proses *increment* saat ini dipindah ke fungsi induk (pemanggil), yaitu di dalam argumen `onClick` pada fungsi `onCreate`.

```

Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            StateHoistingTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    var number by remember { mutableStateOf(0) }
                    CounterScreen(Modifier.padding( all = 32.dp),
                        number = number,
                        label = "Increment",
                        onClick = { number++ }
                    )
                }
            }
        }
    }
}

```

- c. *State* sekarang dideklarasikan di dalam `onCreate()`, di luar dan sebelum pemanggilan fungsi `CounterScreen`.

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:
 - a. Label tombol "Increment" menjadi "Double"
 - b. Nilai awal variabel `number` dari 0 menjadi 1.
 - c. Ubah fungsi/lambda `onClick` pada pemanggilan Composable `CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
 - d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

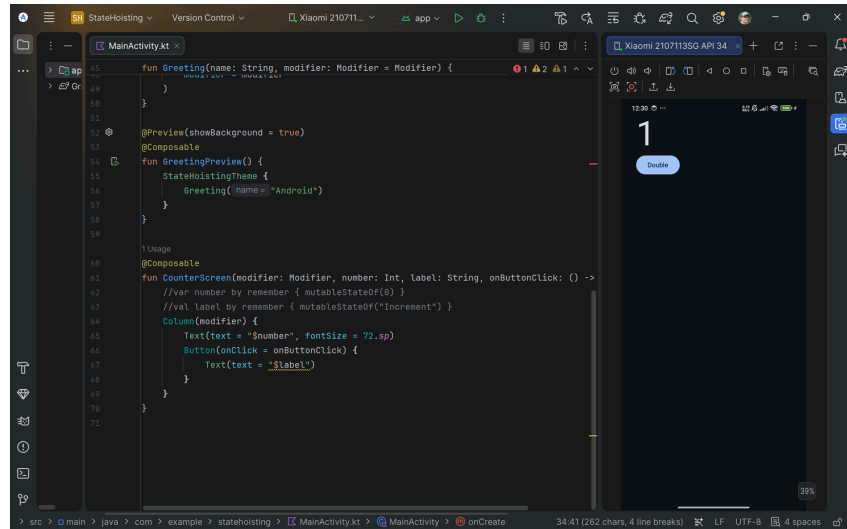
```

JavaScript
var number by remember { mutableStateOf(1) }
CounterScreen(Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onClick = { number *= 2 }
)

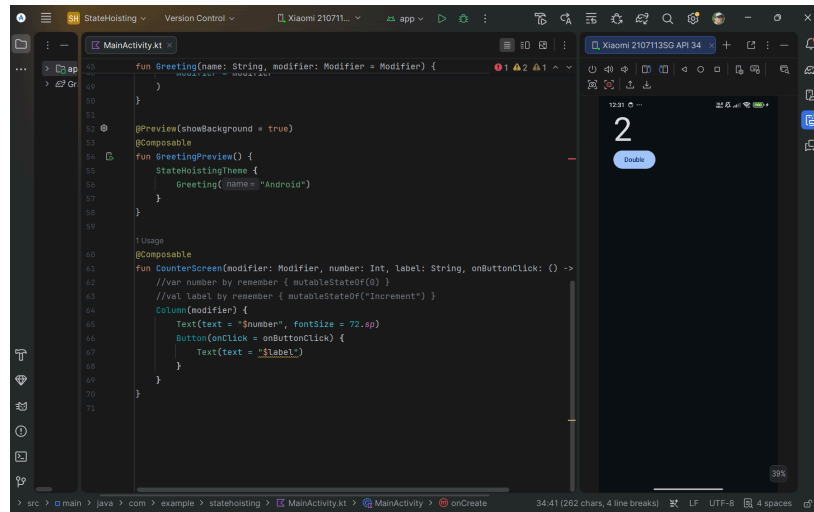
```

6. Jalankan program,
 - a. Perhatikan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.
 - b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?
 - c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable `CounterScreen` itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

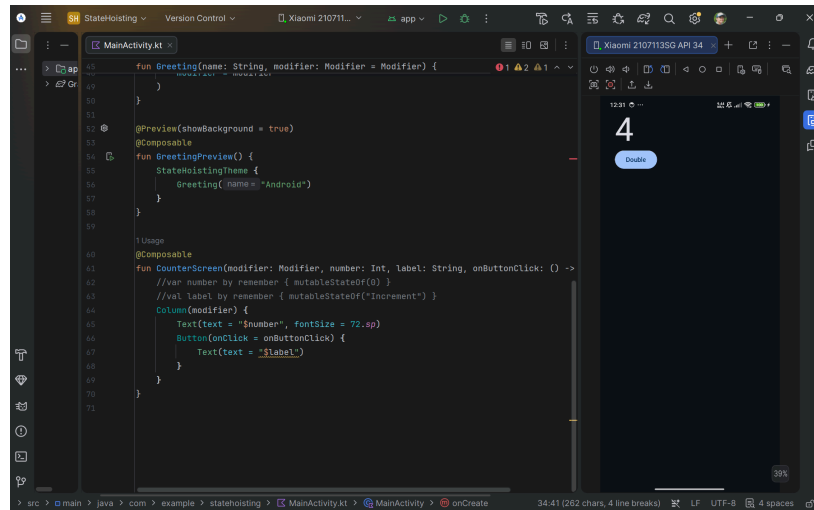
a. Sebelum



b. Diklik 1 kali



c. Diklik 2 kali



7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Tugas Praktik
Lembar Kerja Praktik

Pra-Percobaan

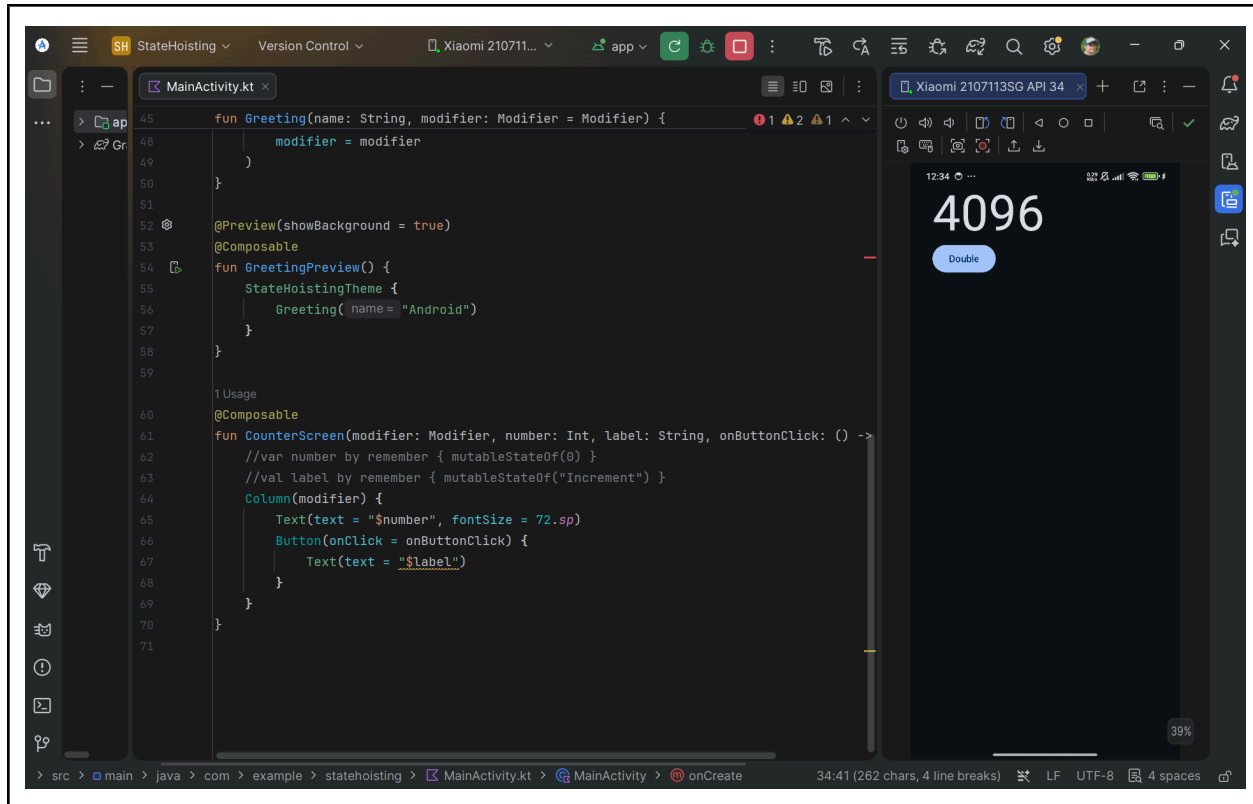
- The screenshot displays the Android Studio IDE with the following components:

 - Top Bar:** Shows the project name "StateHoisting", "Version Control", and the device "Xiaomi 2107113SG API 34".
 - Left Panel:** Displays the project structure with "MainActivity.kt" selected.
 - Main Editor:** Contains the Kotlin code for `MainActivity.kt`. The code defines a `Greeting` function, a `GreetingPreview` function, and a `CounterScreen` function. The `CounterScreen` function uses `remember` and `mutableStateOf` to manage state, and `Column` to layout the UI elements.
 - Right Panel:** Shows a preview of the app interface. The preview displays a large number "1" and a "Double" button. The status bar at the top of the preview shows the time "12:34" and various system icons.
 - Bottom Bar:** Displays the file explorer with the path `src > main > java > com > example > statehoisting > MainActivity.kt`. The bottom right corner shows the file size "34:41 (262 chars, 4 line breaks)" and the encoding "UTF-8".

```
45 fun Greeting(name: String, modifier: Modifier = Modifier) {
46     modifier = modifier
47 }
48
49 @Preview(showBackground = true)
50 @Composable
51 fun GreetingPreview() {
52     StateHoistingTheme {
53         Greeting( name = "Android")
54     }
55 }
56
57 1 Usage
58 @Composable
59 fun CounterScreen(modifier: Modifier, number: Int, label: String, onClick: () ->
60     //var number by remember { mutableStateOf(0) }
61     //val label by remember { mutableStateOf("Increment") }
62     Column(modifier) {
63         Text(text = "$number", fontSize = 72.sp)
64         Button(onClick = onClick) {
65             Text(text = "$label")
66         }
67     }
68 }
69
70
71
```

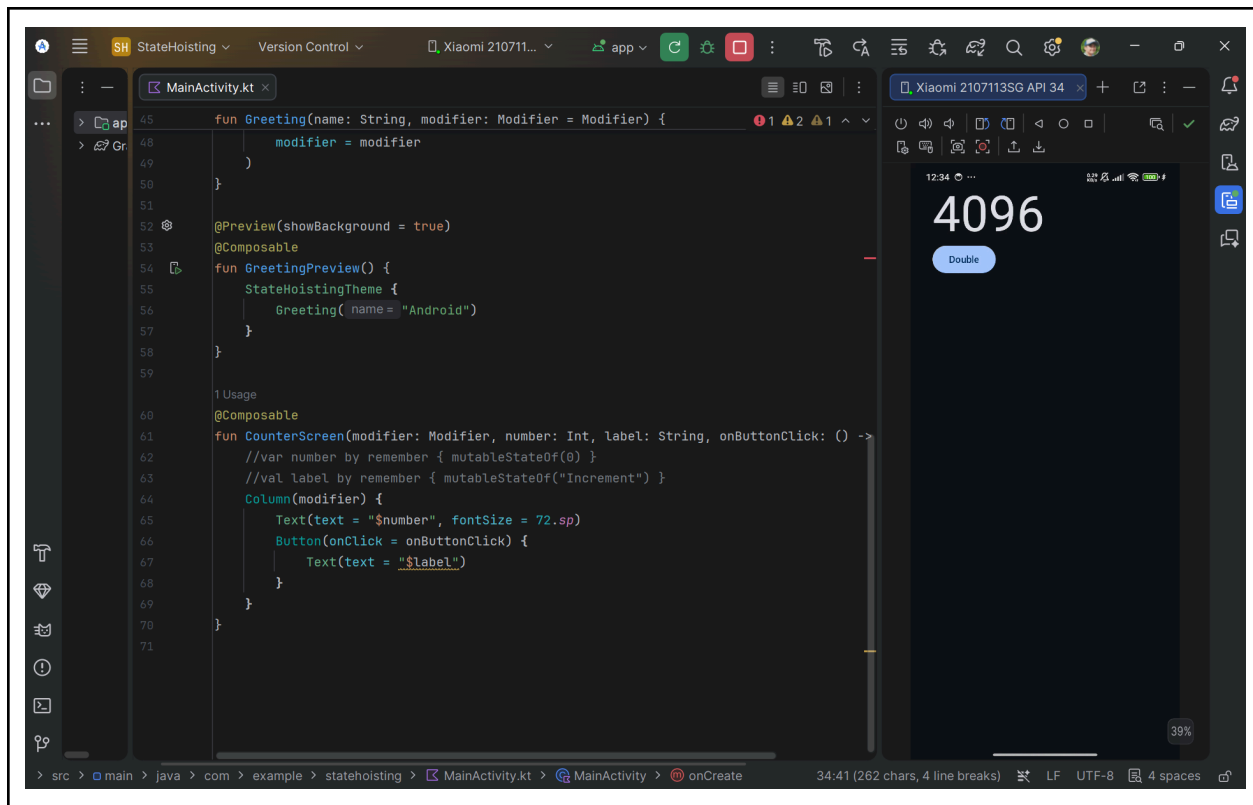
Install successfully finished in 928 ms

2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel number).



Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.
 - b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

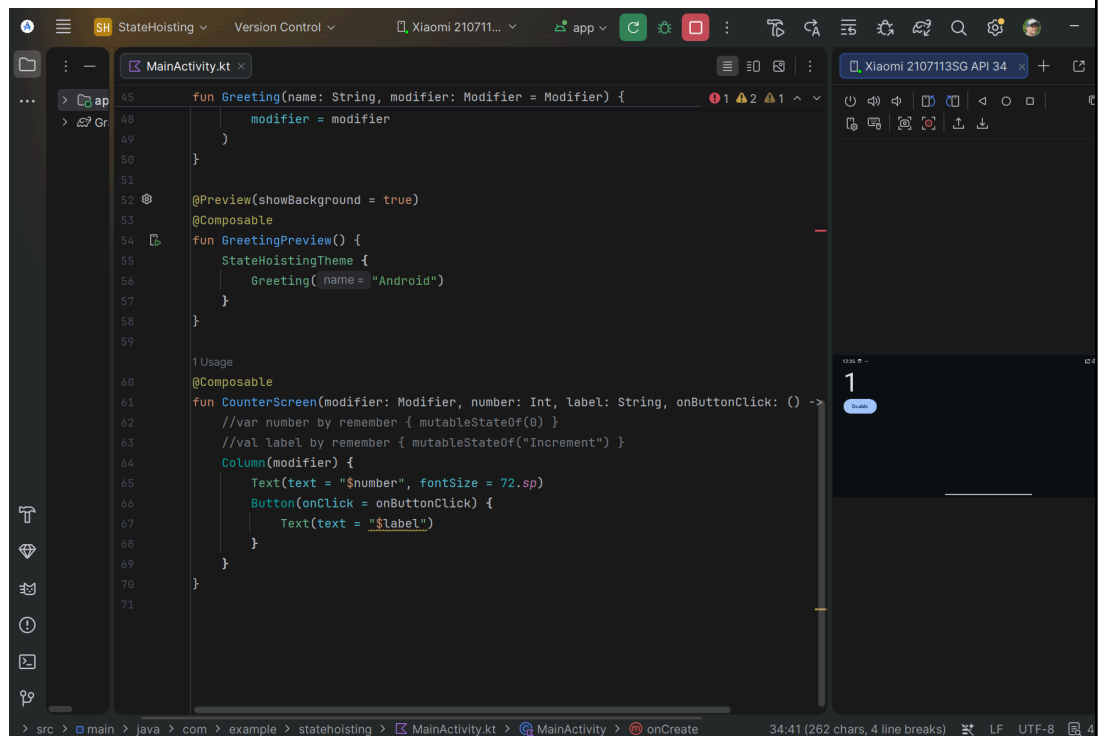
```
var number by remember { mutableStateOf(1) }
```

menjadi:

```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan *import dependency* atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela *editor* Android Studio).

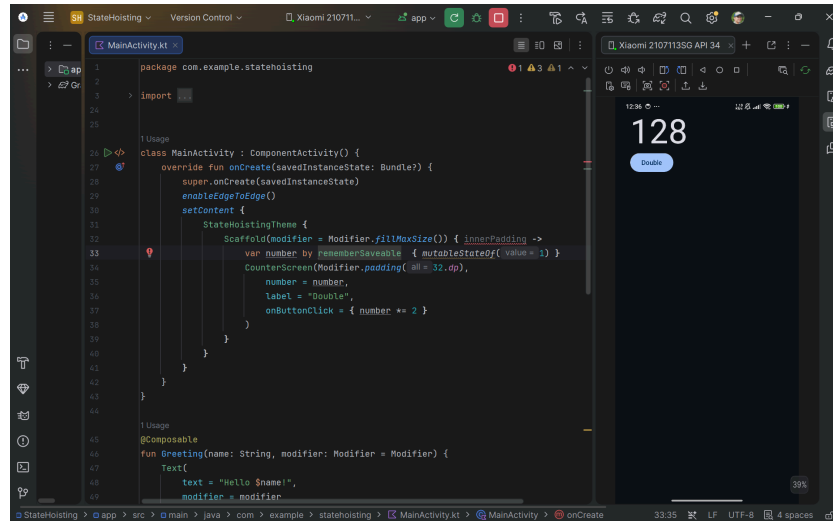
Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



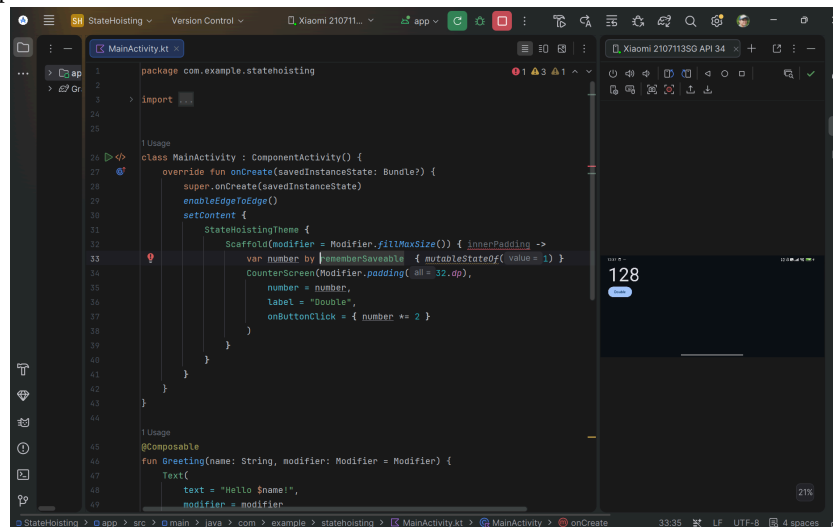
- a.
- b. Angka akan kembali di-reset ke nilai inisialisasi awalnya (1). Hal ini disebabkan karena struktur *remember* biasa tidak mempertahankan *state* ketika *Activity* dihancurkan dan dibuat ulang karena putaran layar (perubahan konfigurasi).

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi *landscape*.
 - b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?
 - c. Apa fungsi dari konstruk *rememberSavable* pada kode program Android yang membedakannya dari konstruk *remember*?

- a. Portrait :



Landscape :



- b. Angka yang ditampilkan akan tetap mempertahankan nilai terakhirnya sebelum layar dirotasi. Perubahan struktur ke *rememberSaveable* menjamin data aman dari putaran orientasi perangkat
- c. **Apa fungsi dari konstruk *rememberSaveable*?** Fungsi utamanya adalah untuk secara otomatis menyimpan tipe data *state* ke dalam sebuah objek *Bundle*. Ini membuat *state* bertahan dari recomposisi sekaligus dari pembuatan ulang proses *Activity*.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk *rememberSaveable* ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Penggunaan yang sangat tepat adalah untuk menyimpan status teks yang sedang diketik di dalam Form isian, menyimpan status kotak centang (*checkbox*), atau menahan posisi gulir layar (*scroll*) pengguna. Manfaat utamanya adalah mencegah hilangnya interaksi masukan data apabila perangkat dimiringkan, sehingga pengalaman pengguna tetap baik.

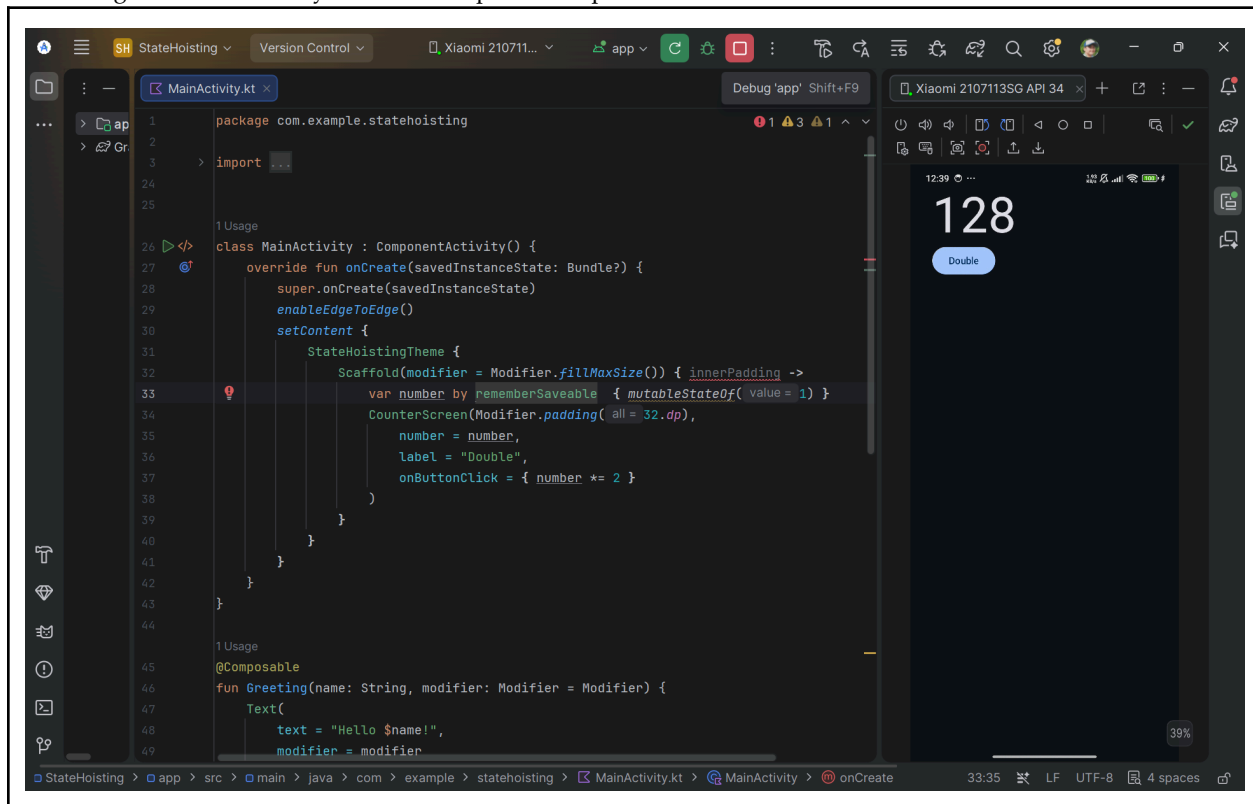
Tugas Praktik

Lembar Kerja Praktik

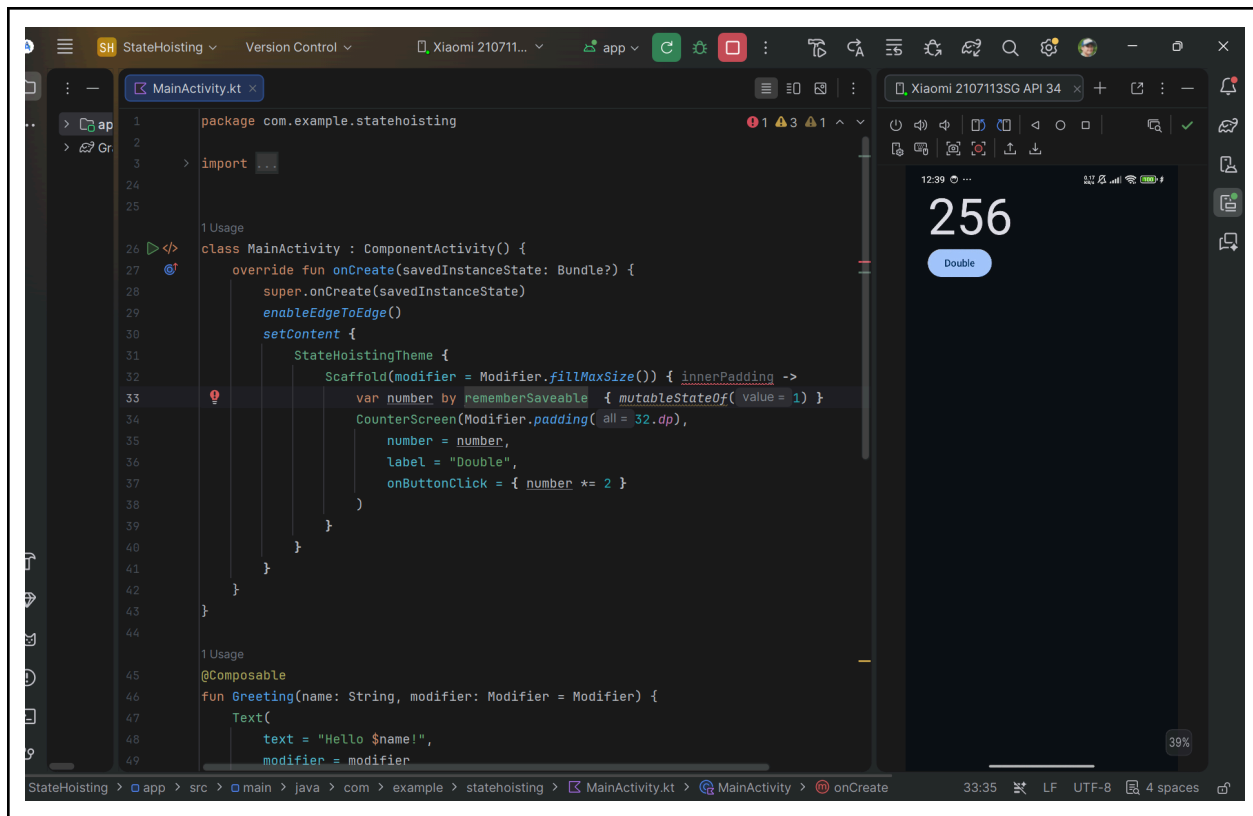
1.1.3 LaunchedEffect

Pra-Percobaan 1

1. Untuk percobaan ini, gunakan Project Android Studio yang sama dengan kode program percobaan Perubahan Konfigurasi setelah menyelesaikan tahap nomor 4 percobaan tersebut.



3. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel `number`).



4. Mengaitkan composable `SnackBar` di dalam `Scaffold`.

Di dalam fungsi `onCreate()` dan di dalam lambda `Theme{}` sebelum pemanggilan composable `Scaffold` dilakukan, tambahkan deklarasi host `state` untuk menyimpan `state` composable `SnackBar` yang akan digunakan untuk menampilkan pesan:

```
val snackbarHostState = remember { SnackbarHostState() }
```

Kemudian ubah bagian pemanggilan fungsi Composable `Scaffold` di dalam fungsi `onCreate()` yang sebelumnya:

```
Scaffold(modifier = Modifier.fillMaxSize())
```

menjadi:

```
Scaffold(
    modifier = Modifier.fillMaxSize(),
    snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
)
```

```
1 package com.example.statehoisting
2
3 > import ...
4
5 1 Usage
6
7 class MainActivity : AppCompatActivity() {
8     override fun onCreate(savedInstanceState: Bundle?) {
9         super.onCreate(savedInstanceState)
10        enableEdgeToEdge()
11        setContent {
12            StateHoistingTheme {
13                val snackbarHostState = remember { SnackbarHostState() }
14                Scaffold(
15                    modifier = Modifier.fillMaxSize(),
16                    snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
17                ) {
18                    { innerPadding ->
19                        var number by rememberSaveable { mutableStateOf(value = 1) }
20                        CounterScreen(Modifier.padding(all = 32.dp),
21                            number = number,
22                            label = "Double",
23                            onClick = { number *= 2 }
24                        )
25                    }
26                }
27            }
28        }
29    }
30}
```

5. Menerapkan fitur Side-effect dengan LaunchedEffect. Dengan posisi kursor tetap berada di dalam lambda `Scaffold`, setelah pemanggilan composable `CounterScreen` tambahkan kode program *side-effect* berikut ini:

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```

```

class MainActivity : ComponentActivity() {
    fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        window.setStatusBarColor(Color.TRANSPARENT)
        val theme = ThemeStateHoistingTheme {
            val snackbarHostState = remember { SnackbarHostState() }
            Scaffold(
                modifier = Modifier.fillMaxSize(),
                snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
            ) { innerPadding ->
                var number by rememberSaveable { mutableStateOf( value = 1) }
                CounterScreen(Modifier.padding( all = 32.dp),
                    number = number,
                    label = "Double",
                    onClick = { number *= 2 }
                )
                LaunchedEffect( key1 = number) {
                    snackbarHostState.showSnackbar( message = "Number $number is shown!")
                }
            }
        }
    }
}

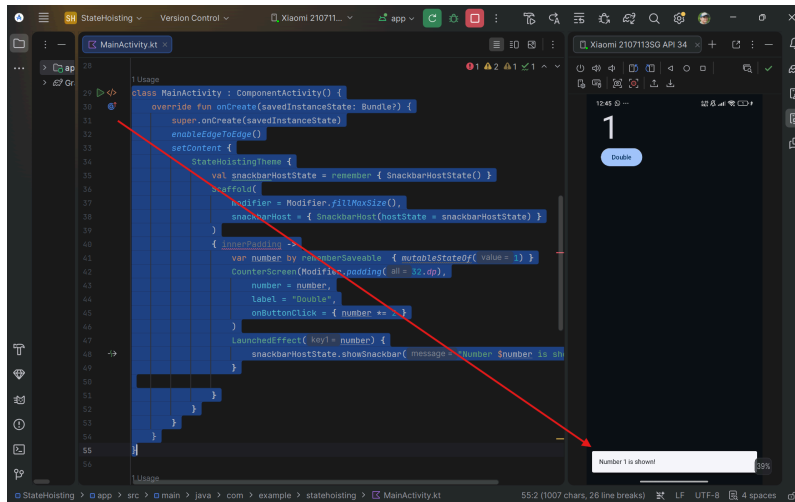
```

6. Lakukan *import* semua *dependency* yang diperlukan dari perubahan kode program yang telah dilakukan (dengan melakukan *refactoring* melalui Alt-Enter pada bagian yang ditandai merah di dalam jendela editor Android Studio) agar perubahan kode program yang dilakukan dapat di-*compile* dan dijalankan dengan baik. Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi onCreate(). Tidak ada perubahan yang dilakukan di dalam fungsi composable CounterScreen().

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.statehoisting.ui.theme.StateHoistingTheme
import androidx.compose.runtime.remember
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.material3.Button
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

Aktivitas Percobaan

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?
 - b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?
 - c. Apa yang memicu pesan pada *SnackBar* tersebut ditampilkan? Pada bagian kode program yang mana?



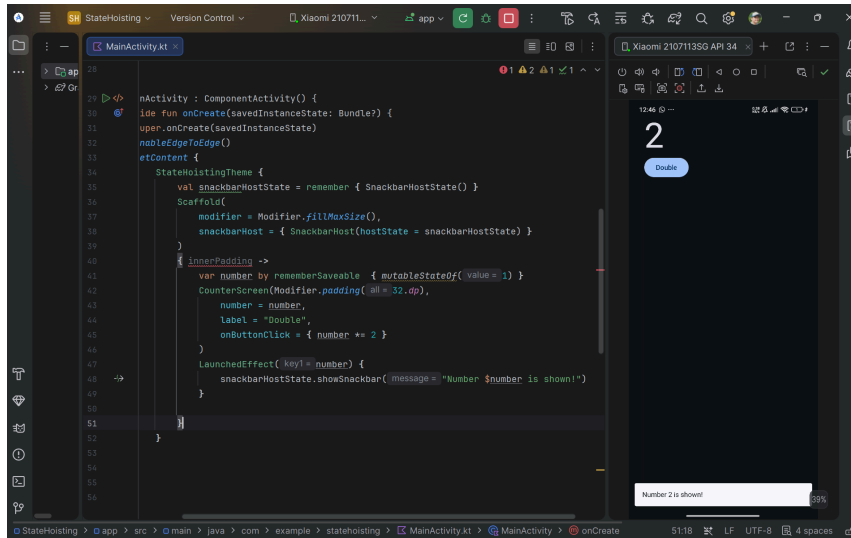
-
- Karena komponen *LaunchedEffect* dikonfigurasi untuk menjalankan otomatis *coroutine* di dalamnya tepat ketika *Composable* tersebut pertama kali dimuat (masuk ke dalam komposisi).
- Pemicu awalnya adalah deklarasi eksekusi *LaunchedEffect(number)* yang baru ditambahkan ke dalam kerangka fungsi *onCreate()*.

```

class MainActivity : AppCompatActivity() {
    fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView()
        stateHoistingTheme {
            val snackbarHostState = remember { SnackbarHostState() }
            Scaffold(
                modifier = Modifier.fillMaxSize(),
                snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
            ) {
                { innerPadding ->
                    var number by rememberSaveable { mutableStateOf( value = 1 ) }
                    CounterScreen(Modifier.padding( all = 32.dp ),
                        number = number,
                        label = "Double",
                        onClick = { number *= 2 }
                    )
                }
                LaunchedEffect( key1 = number ) {
                    snackbarHostState.showSnackBar( message = "Number $number is shown!" )
                }
            }
        }
    }
}

```

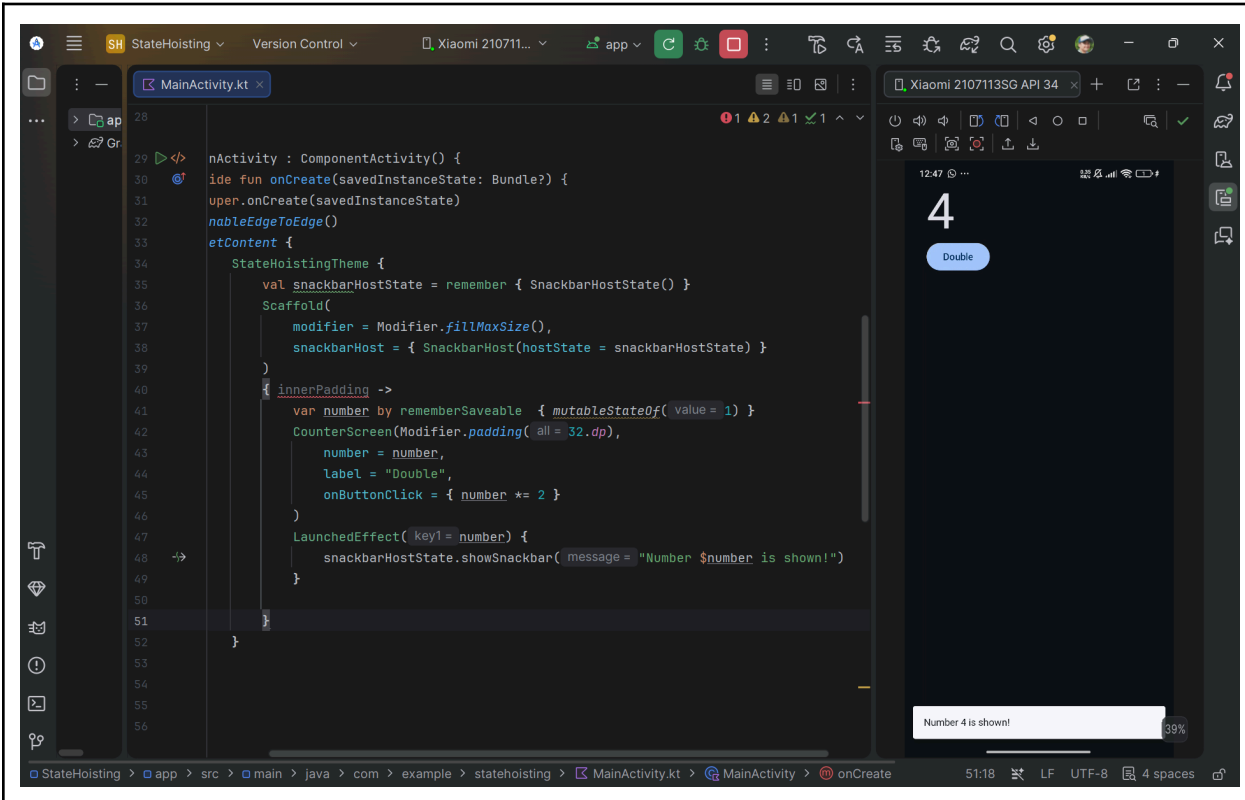
- Klik button "Double" satu kali,
 - Apa yang terjadi?
 - Mengapa hal tersebut terjadi? Jelaskan!



-
- Pesan *SnackBar* yang baru akan muncul menampilkan angka hasil perhitungan terbaru. *LaunchedEffect* mengawasi parameter nilai kuncinya (yakni *number*). Saat *state* variabel *number* berubah karena tombol diklik, *coroutine* lama dibatalkan dan diluncurkan ulang menggunakan variabel terbaru.

Pra-Percobaan 2

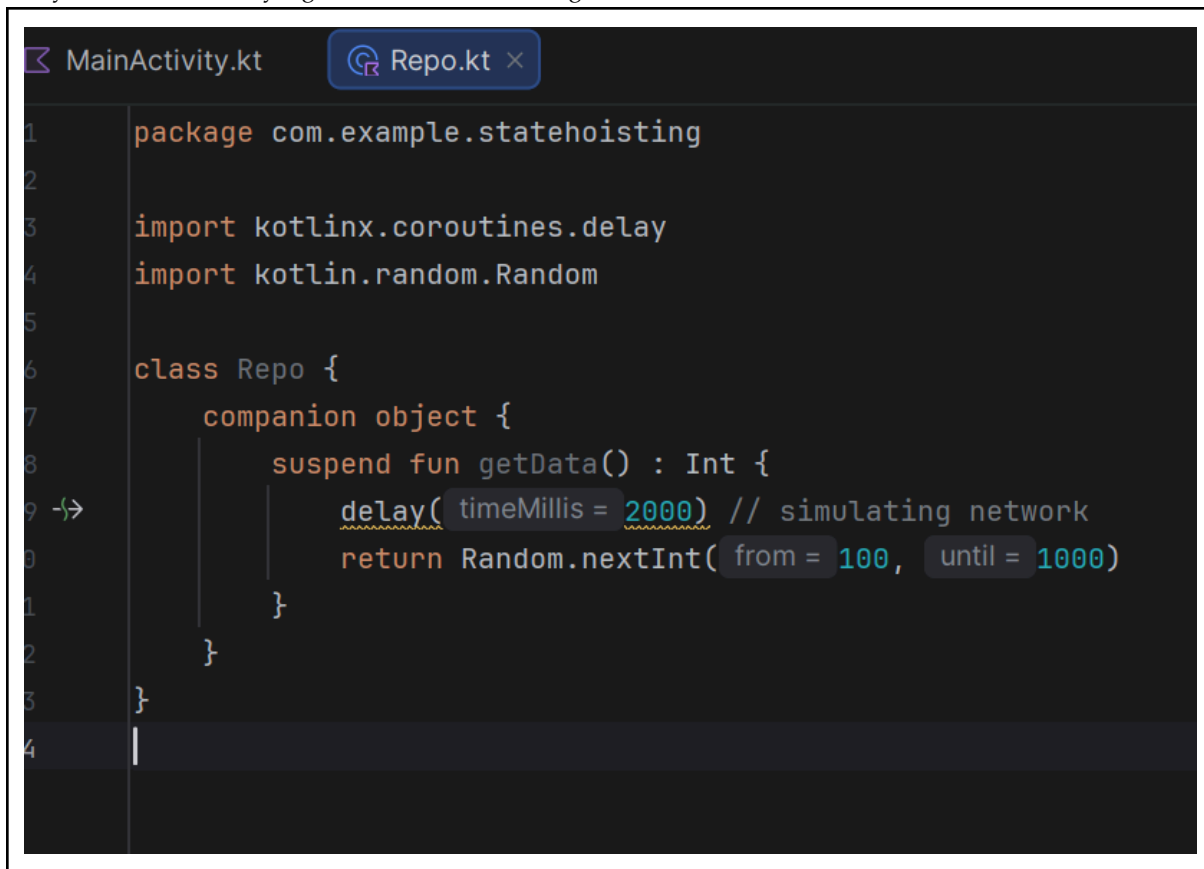
- Untuk percobaan ini, lanjutkan Project Android Studio yang sama dengan kode program percobaan Side-effect sebelumnya.



4. Di dalam package yang sama dengan class `MainActivity`, buat sebuah class Kotlin baru bernama `Repo` (file: `Repo.kt`) dengan struktur class dan fungsi static `getData()` seperti pada kode program berikut ini:

```
class Repo {  
    companion object {  
        suspend fun getData() : Int {  
            delay(2000) // simulating network  
            return Random.nextInt(100, 1000)  
        }  
    }  
}
```

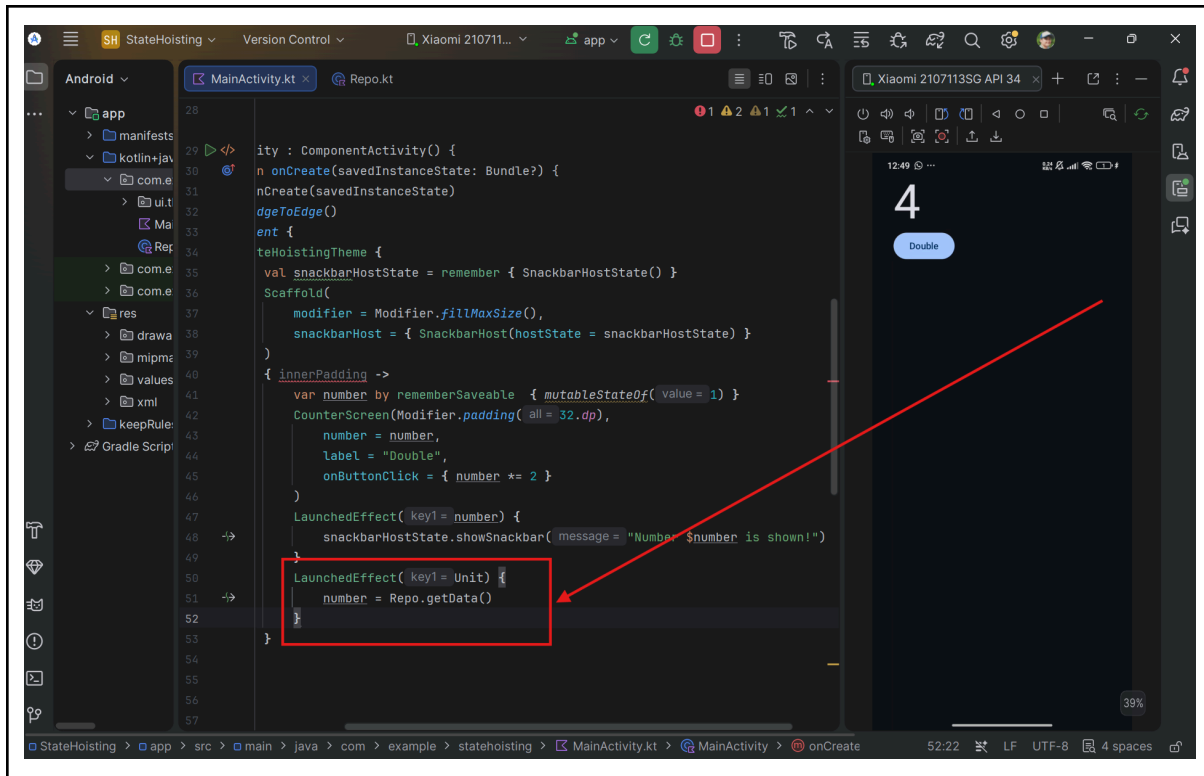
Lakukan import dependency untuk fungsi `delay()` dan class `Random`. Fungsi `delay(2000)` ini digunakan untuk menyimulasikan waktu yang dibutuhkan untuk mengambil data melalui koneksi internet.

A screenshot of an IDE window showing the `Repo.kt` file. The window has two tabs: `MainActivity.kt` and `Repo.kt`. The `Repo.kt` tab is active. The code in the editor is as follows:

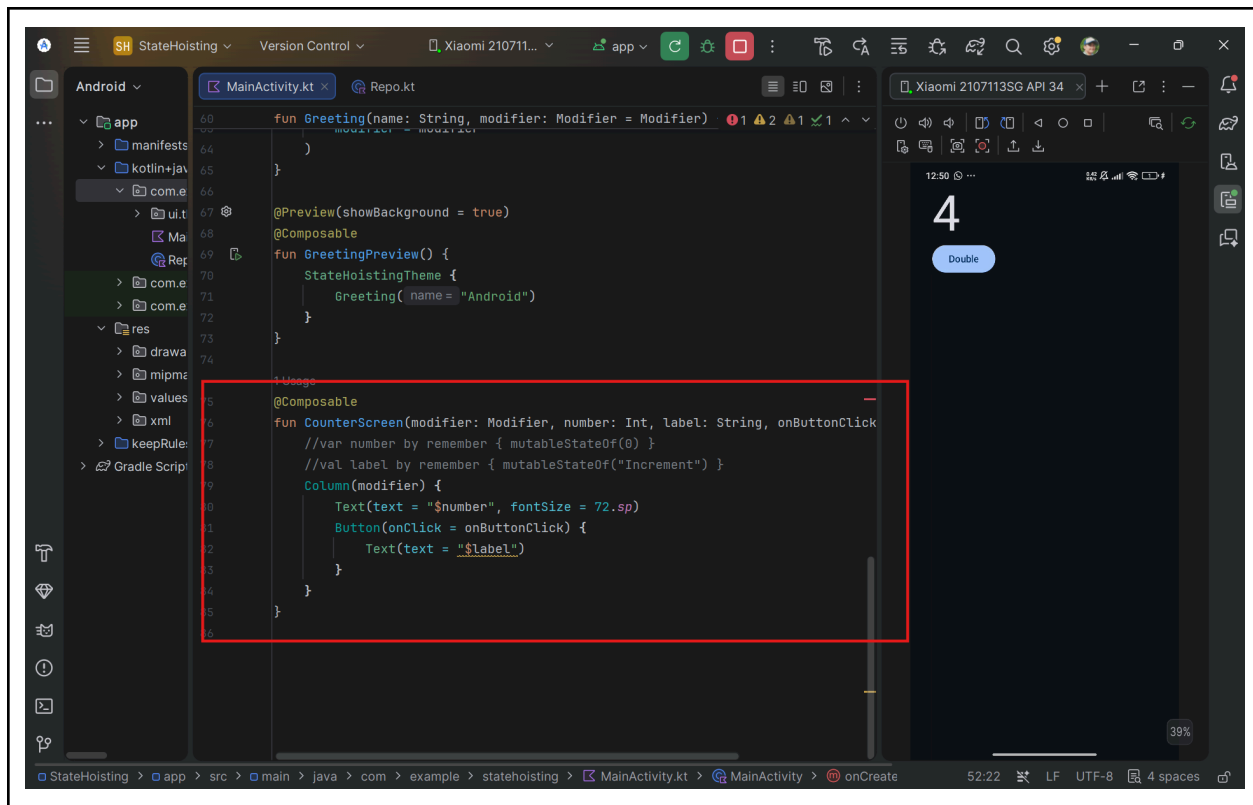
```
1 package com.example.statehoisting  
2  
3 import kotlinx.coroutines.delay  
4 import kotlin.random.Random  
5  
6 class Repo {  
7     companion object {  
8         suspend fun getData() : Int {  
9             delay(timeMillis = 2000) // simulating network  
10            return Random.nextInt(from = 100, until = 1000)  
11        }  
12    }  
13 }  
14 |
```

5. Di dalam fungsi `onCreate()`, setelah `LaunchedEffect` pada percobaan sebelumnya, tambahkan deklarasi fungsi `LaunchedEffect` baru berikut ini:

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

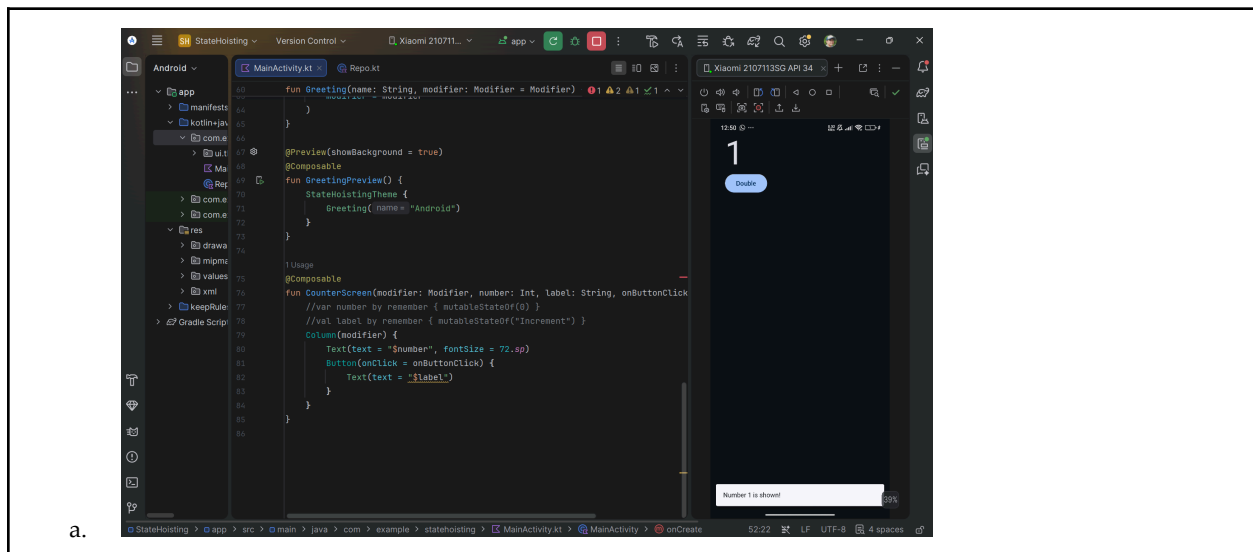


7. Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi onCreate(). Tidak ada perubahan yang dilakukan di dalam fungsi composable CounterScreen().

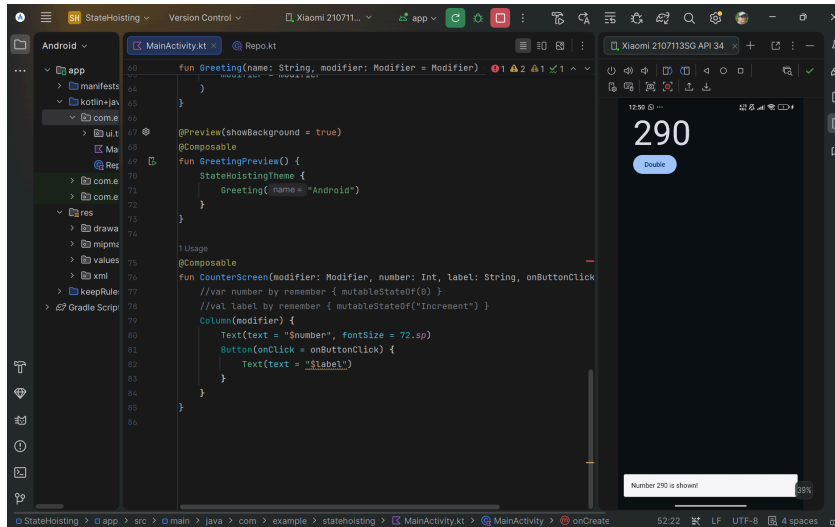


Aktivitas Percobaan

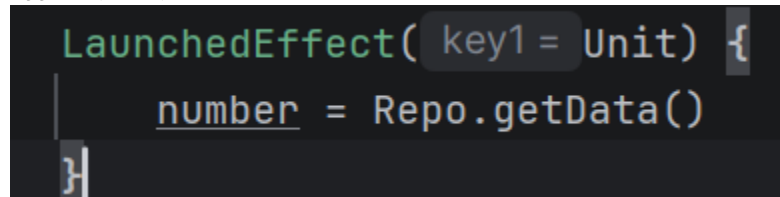
1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?
 - b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?
 - c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?



a.



- b.
- c. Pemanggilan fungsi asinkron `Repo.getData()` yang diletakkan di dalam kerangka blok kode `LaunchedEffect(Unit)`



6. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,
 - a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?
 - b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

- a. Blok program di dalam `LaunchedEffect(state)` akan terus dibatalkan dan dieksekusi ulang secara repetitif setiap kali nilai `state` kuncinya mendeteksi pergantian. Sebaliknya, deklarasi struktur `LaunchedEffect(Unit)` dipastikan hanya akan dipicu murni satu kali eksekusi.
- b. Kedua varian tersebut akan secara otomatis dipanggil pertama kali saat komponen menjejaki tahapan masuk komposisi. Namun, hanya `LaunchedEffect(state)` yang akan reaktif bekerja berulang sepanjang siklus hidup layar apabila mendeteksi adanya mutasi pada `state` observasinya.